EXPT NO: 2 DATE:27th July,2026	COMPARATIVE STUDY OF ACTIVATION FUNCTIONS AND OPTIMIZATION ALGORITHMS IN DEEP LEARNING

PRE-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. **What is the purpose of an activation function in an Artificial Neural Network?**
   An activation function determines whether a neuron should be activated by transforming its input into an output. It introduces non-linearity, allowing the neural network to learn complex patterns and relationships in data. Without activation functions, an ANN would behave like a simple linear model and fail to solve complex tasks such as image recognition or classification.

2. **Differentiate between the Sigmoid, Tanh, and ReLU activation functions based on their output range and mathematical properties.**
   The Sigmoid function outputs values between 0 and 1, making it suitable for binary classification. Tanh produces outputs between -1 and 1, resulting in better-centered data and faster convergence. ReLU outputs values from 0 to infinity, is computationally efficient, reduces the vanishing gradient problem, and is commonly used in hidden layers.

3. **Why is an optimization algorithm required during neural network training?**
   An optimization algorithm is used to minimize the loss function by adjusting the network's weights and biases during training. It helps the model learn from errors through backpropagation and improves prediction accuracy. Efficient optimizers, such as Adam or SGD, speed up convergence and ensure the neural network reaches an optimal solution.

4. **What is the difference between Batch Gradient Descent, Stochastic Gradient Descent (SGD), and Mini-Batch Gradient Descent?**
   Batch Gradient Descent updates weights after processing the entire dataset, making it stable but slow. Stochastic Gradient Descent updates weights after each training sample, providing faster learning but with noisy updates. Mini-Batch Gradient Descent processes small groups of samples, balancing speed, memory efficiency, and stable convergence, making it the most widely used approach.

5. **Explain the role of Google Colab and GitHub in deep learning experiment management.**
   Google Colab provides a cloud-based environment with free GPU and TPU support, allowing users to develop, train, and test deep learning models without local setup. GitHub helps manage project versions, store code, collaborate with team members, track changes, and maintain experiment history, ensuring organized and reproducible deep learning development.

AIM

To analyze the impact of different activation functions and optimization algorithms on the performance of Artificial Neural Networks (ANNs) and to learn best practices for managing deep learning experiments using cloud-based tools and version control.

SOFTWARE REQUIREMENTS

- Python
- TensorFlow or PyTorch
- Google Colab
- GitHub
- Matplotlib

OBJECTIVES

Students will be able to:

- Understand the role of activation functions in neural networks.
- Visualize and compare commonly used activation functions.
- Evaluate the performance of different activation functions in ANN models.
- Analyze various gradient-based optimization algorithms.
- Compare optimizer convergence and model accuracy.
- Learn experiment management using Google Colab, Google Drive, and GitHub.
- Maintain reproducible deep learning experiments using version control.

PROCEDURE

PART A: VISUALIZATION OF ACTIVATION FUNCTIONS

Implement and plot the following activation functions:

- Sigmoid
- Tanh
- ReLU

Compare their:

- Output range
- Saturation regions
- Gradient behavior
- Computational efficiency
- Typical applications

PART B: PERFORMANCE COMPARISON OF ACTIVATION FUNCTIONS

Train identical ANN architectures using:

- Sigmoid
- Tanh
- ReLU

Record and compare:

- Training accuracy
- Validation accuracy
- Training loss
- Validation loss
- Number of epochs required for convergence

Analyze which activation function performs best and justify the observations.

PART C: COMPARISON OF OPTIMIZATION ALGORITHMS

Train the same ANN using:

- Stochastic Gradient Descent (SGD)
- Momentum
- RMSProp
- Adam

Compare:

- Training loss
- Validation loss
- Convergence speed
- Final training accuracy
- Final validation accuracy

Visualize:

- Loss vs Epoch
- Accuracy vs Epoch

Discuss which optimizer converges faster and produces better generalization.

PART D: DEEP LEARNING EXPERIMENT MANAGEMENT

Perform the following tasks:

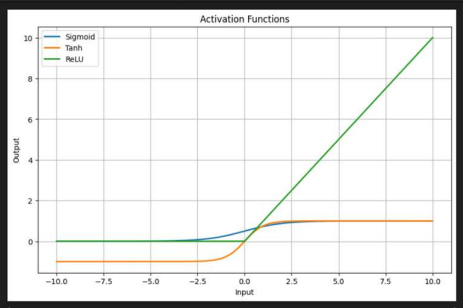
- Create a Google Colab notebook.
- Mount Google Drive.
- Save notebooks and trained models.
- Create a GitHub repository.
- Upload notebooks to GitHub.
- Commit changes and maintain version history using Git.
- Document the experiment using a README.md file.

EXPECTED OUTCOME

Students will understand how activation functions and optimization algorithms influence neural network learning and performance. They will also gain practical experience in organizing, documenting, and maintaining reproducible deep learning experiments using cloud platforms and Git-based version control.

Pradeep Manikandan D 24BAD088

```
# Import Libraries
import numpy as np
import matplotlib.pyplot as plt
# Input values
x = np.linspace(-10, 10, 500)

# Activation Functions
sigmoid = 1 / (1 + np.exp(-x))
tanh = np.tanh(x)
relu = np.maximum(0, x)

# Plot
plt.figure(figsize=(10,6))
plt.plot(x, sigmoid, label='Sigmoid', linewidth=2)
plt.plot(x, tanh, label='Tanh', linewidth=2)
plt.plot(x, relu, label='ReLU', linewidth=2)

plt.title("Activation Functions")
plt.xlabel("Input")
plt.ylabel("Output")
plt.legend()
plt.grid(True)
plt.show()
```



Performance Comparison of Activation Functions

```
# Import Libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
```

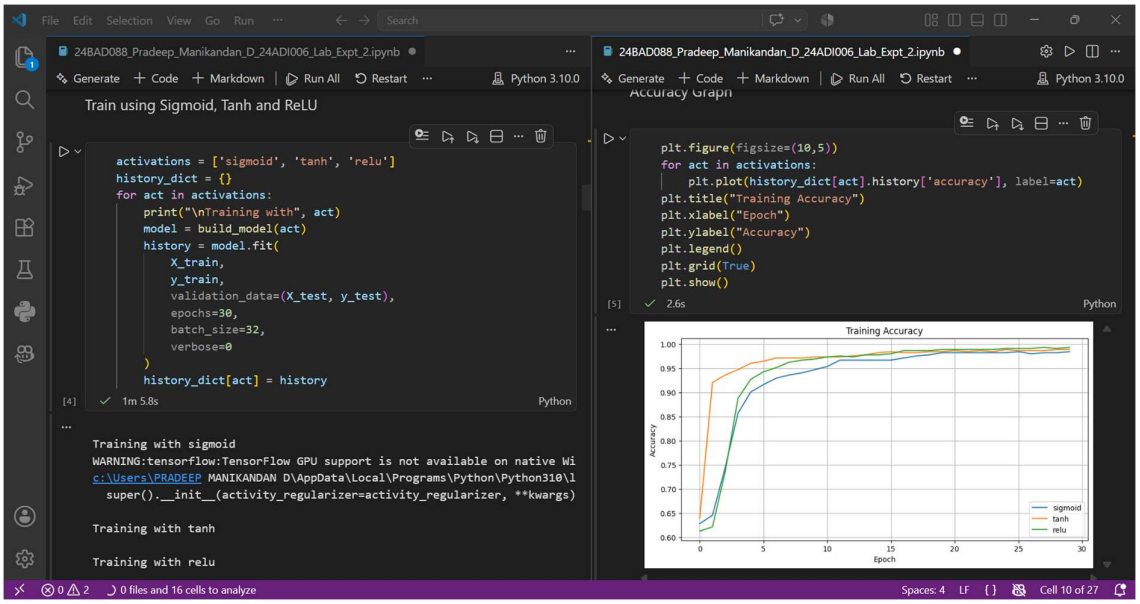
Performance Comparison of Activation Functions

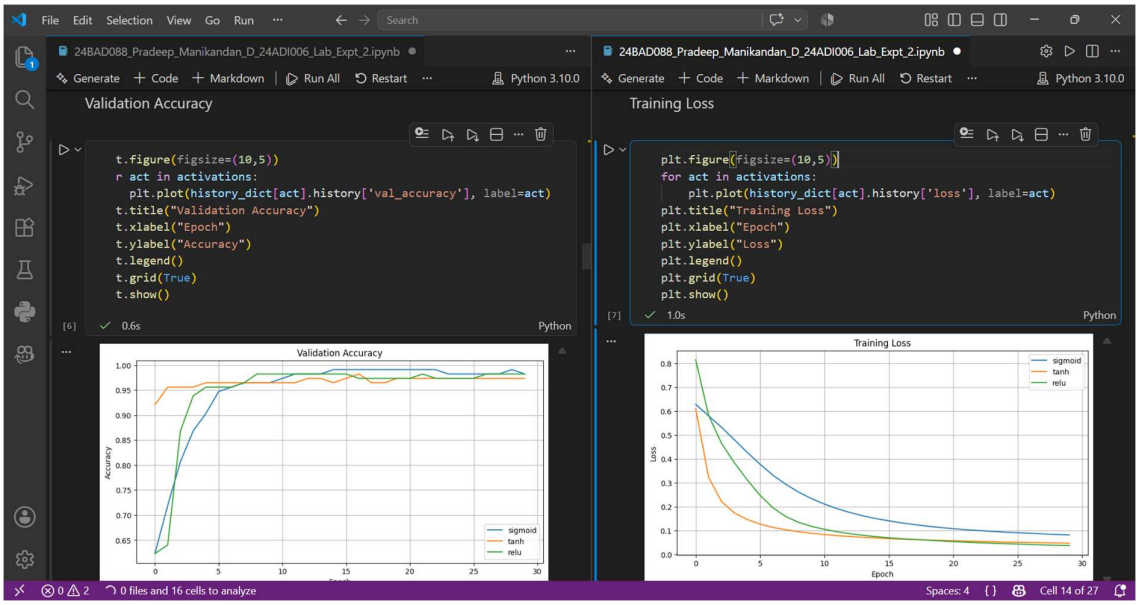
```
# Import Libraries
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# Load Dataset
data = load_breast_cancer()
X = data.data
y = data.target
# Split Data
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
# Standardization
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

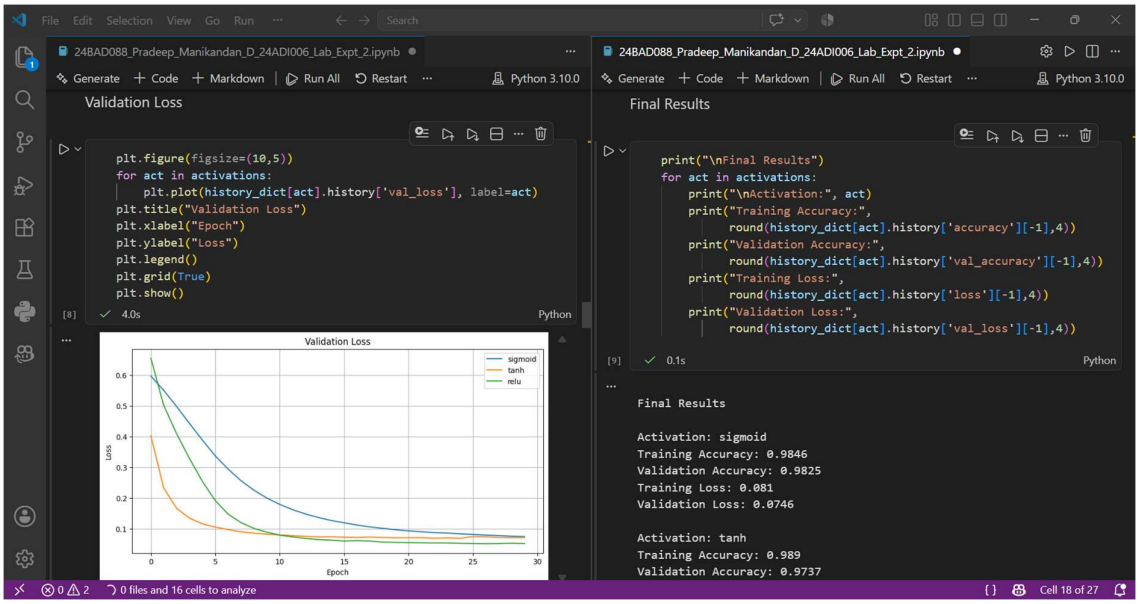
function to build an ANN

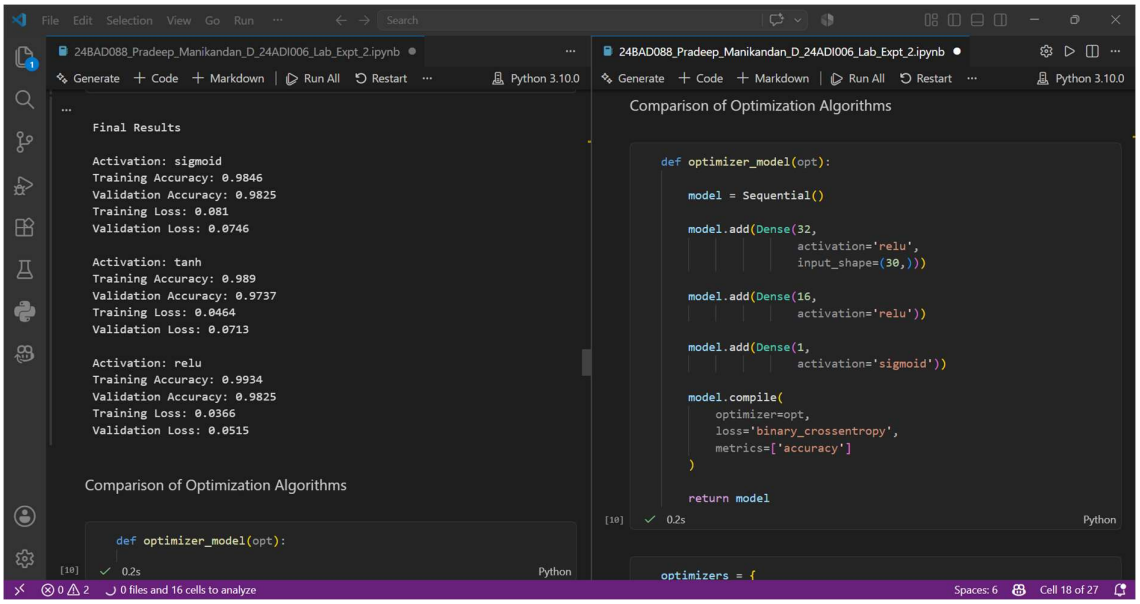
```
def build_model(activation):

    model = Sequential()

    model.add(Dense(32,
                    activation=activation,
                    input_shape=(30,)))

    model.add(Dense(16,
                    activation=activation))

    model.add(Dense(1,
                    activation='sigmoid'))

    model.compile(
        optimizer='adam',
        loss='binary_crossentropy',
        metrics=['accuracy']
    )

    return model
```

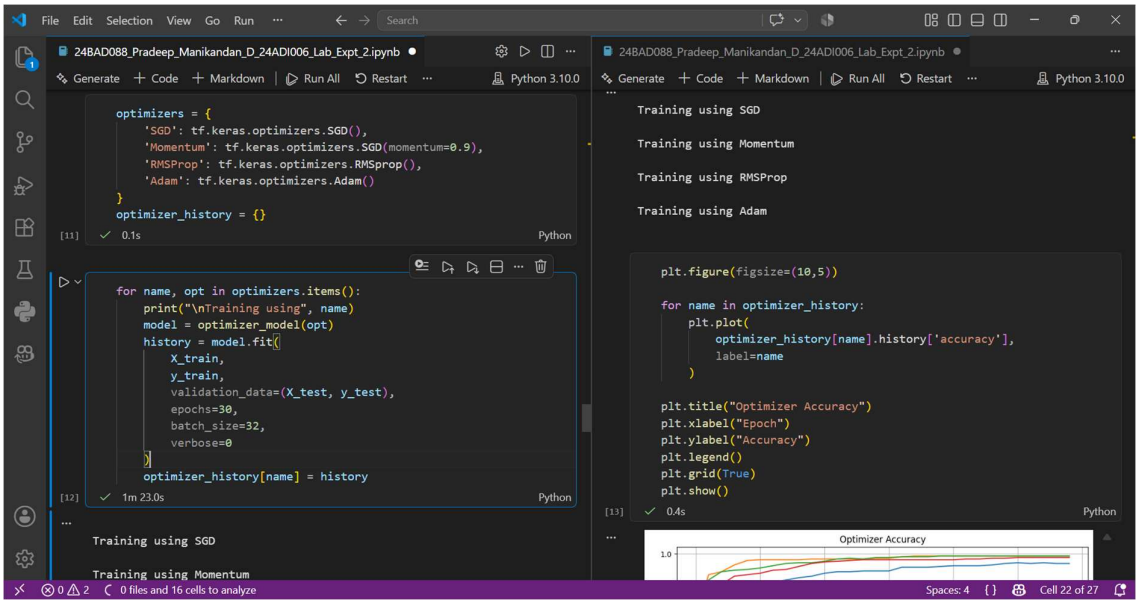
Train using Sigmoid, Tanh and ReLU

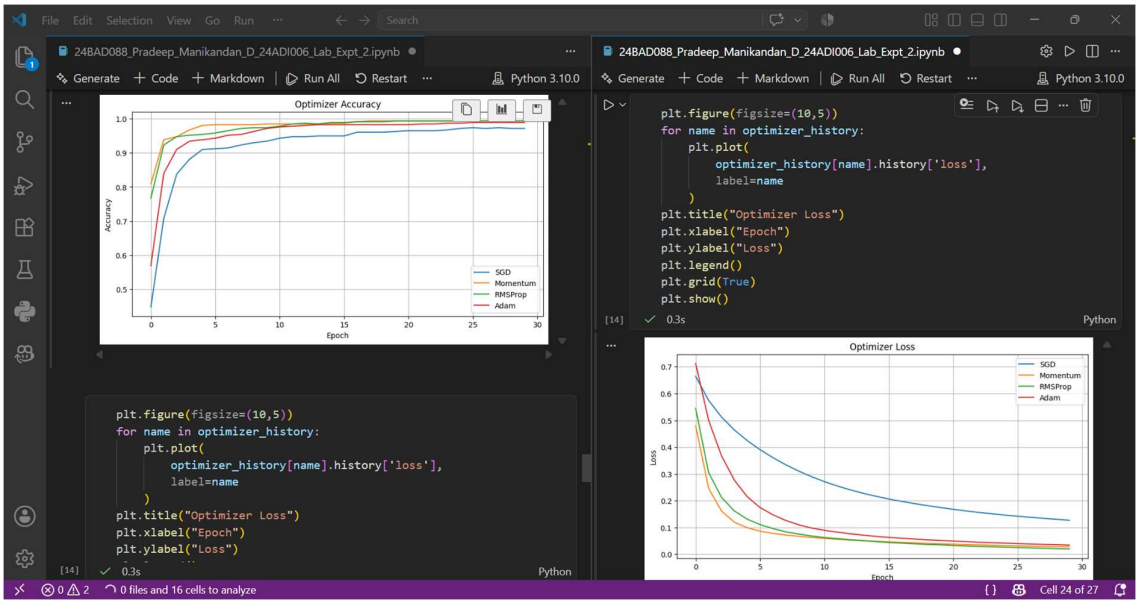

Train using Sigmoid, Tanh and ReLU
activations = ['sigmoid', 'tanh', 'relu']
history_dict = {}
for act in activations:
print("\nTraining with", act)
model = build_model(act)
history = model.fit(
X_train,
y_train,
validation_data=(X_test, y_test),
epochs=30,
batch_size=32,
verbose=0
)
history_dict[act] = history
Training with sigmoid
Training with tanh
Training with relu
plt.figure(figsize=(10,5))
for act in activations:
plt.plot(history_dict[act].history['accuracy'], label=act)
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
Training Accuracy
sigmoid
tanh
relu

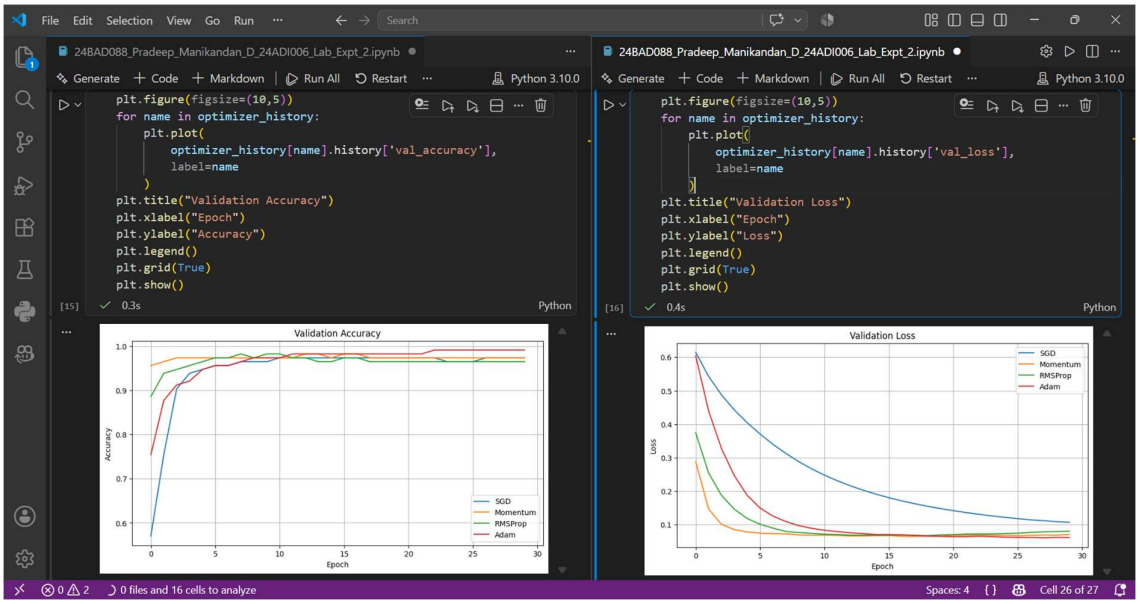

Validation Accuracy
t.figure(figsize=(10,5))
r act in activations:
plt.plot(history_dict[act].history['val_accuracy'], label=act)
t.title("Validation Accuracy")
t.xlabel("Epoch")
t.ylabel("Accuracy")
t.legend()
t.grid(True)
t.show()
Training Loss
plt.figure(figsize=(10,5))
for act in activations:
plt.plot(history_dict[act].history['loss'], label=act)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
sigmoid
tanh
relu

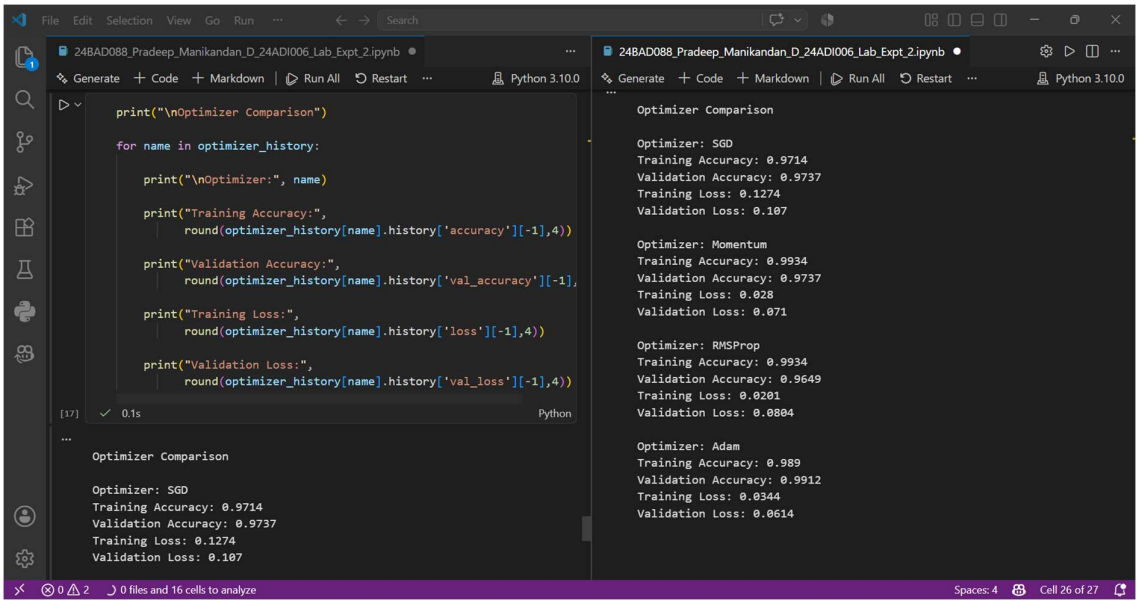

24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
Validation Loss
plt.figure(figsize=(10,5))
for act in activations:
plt.plot(history_dict[act].history['val_loss'], label=act)
plt.title("Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
[8] 4.0s Python
Validation Loss
sigmoid
tanh
relu
Loss
Epoch
Final Results
print("\nFinal Results")
for act in activations:
print("\nActivation:", act)
print("Training Accuracy:",
round(history_dict[act].history['accuracy'][-1],4))
print("Validation Accuracy:",
round(history_dict[act].history['val_accuracy'][-1],4))
print("Training Loss:",
round(history_dict[act].history['loss'][-1],4))
print("Validation Loss:",
round(history_dict[act].history['val_loss'][-1],4))
[9] 0.1s Python
Final Results
Activation: sigmoid
Training Accuracy: 0.9846
Validation Accuracy: 0.9825
Training Loss: 0.081
Validation Loss: 0.0746
Activation: tanh
Training Accuracy: 0.989
Validation Accuracy: 0.9737
0 files and 16 cells to analyze
Cell 18 of 27


24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
Final Results
Activation: sigmoid
Training Accuracy: 0.9846
Validation Accuracy: 0.9825
Training Loss: 0.081
Validation Loss: 0.0746
Activation: tanh
Training Accuracy: 0.989
Validation Accuracy: 0.9737
Training Loss: 0.0464
Validation Loss: 0.0713
Activation: relu
Training Accuracy: 0.9934
Validation Accuracy: 0.9825
Training Loss: 0.0366
Validation Loss: 0.0515
Comparison of Optimization Algorithms
def optimizer_model(opt):
[10] 0.2s Python
Comparison of Optimization Algorithms
def optimizer_model(opt):
model = Sequential()
model.add(Dense(32,
activation='relu',
input_shape=(30,)))
model.add(Dense(16,
activation='relu'))
model.add(Dense(1,
activation='sigmoid'))
model.compile(
optimizer=opt,
loss='binary_crossentropy',
metrics=['accuracy']
)
return model
[10] 0.2s Python
optimizers = {
0 files and 16 cells to analyze
Spaces: 6
Cell 18 of 27


24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
optimizers = {
'SGD': tf.keras.optimizers.SGD(),
'Momentum': tf.keras.optimizers.SGD(momentum=0.9),
'RMSProp': tf.keras.optimizers.RMSprop(),
'Adam': tf.keras.optimizers.Adam()
}
optimizer_history = {}
[11] 0.1s Python
for name, opt in optimizers.items():
print("\nTraining using", name)
model = optimizer_model(opt)
history = model.fit(
X_train,
y_train,
validation_data=(X_test, y_test),
epochs=30,
batch_size=32,
verbose=0
)
optimizer_history[name] = history
[12] 1m 23.0s Python
Training using SGD
Training using Momentum
Training using RMSProp
Training using Adam
plt.figure(figsize=(10,5))
for name in optimizer_history:
plt.plot(
optimizer_history[name].history['accuracy'],
label=name
)
plt.title("Optimizer Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
[13] 0.4s Python
Optimizer Accuracy
0 files and 16 cells to analyze
Spaces: 4
Cell 22 of 27


24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
Optimizer Accuracy
Accuracy
Epoch
SGD
Momentum
RMSProp
Adam
plt.figure(figsize=(10,5))
for name in optimizer_history:
plt.plot(
optimizer_history[name].history['loss'],
label=name
)
plt.title("Optimizer Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
[14] 0.3s Python
Optimizer Loss
0 files and 16 cells to analyze
Cell 24 of 27


24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
plt.figure(figsize=(10,5))
for name in optimizer_history:
plt.plot(
optimizer_history[name].history['val_accuracy'],
label=name
)
plt.title("Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
[15] 0.3s Python
Validation Accuracy
Accuracy
Epoch
SGD
Momentum
RMSProp
Adam
plt.figure(figsize=(10,5))
for name in optimizer_history:
plt.plot(
optimizer_history[name].history['val_loss'],
label=name
)
plt.title("Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
[16] 0.4s Python
Validation Loss
Loss
Epoch
SGD
Momentum
RMSProp
Adam
0 files and 16 cells to analyze
Spaces: 4
Cell 26 of 27


24BAD088_Pradeep_Manikandan_D_24ADI006_Lab_Expt_2.ipynb
Generate
Code
Markdown
Run All
Restart
Python 3.10.0
print("\nOptimizer Comparison")
for name in optimizer_history:
print("\nOptimizer:", name)
print("Training Accuracy:",
round(optimizer_history[name].history['accuracy'][-1],4))
print("Validation Accuracy:",
round(optimizer_history[name].history['val_accuracy'][-1],
print("Training Loss:",
round(optimizer_history[name].history['loss'][-1],4))
print("Validation Loss:",
round(optimizer_history[name].history['val_loss'][-1],4))
[17] 0.1s Python
Optimizer Comparison
Optimizer: SGD
Training Accuracy: 0.9714
Validation Accuracy: 0.9737
Training Loss: 0.1274
Validation Loss: 0.107
Optimizer: Momentum
Training Accuracy: 0.9934
Validation Accuracy: 0.9737
Training Loss: 0.028
Validation Loss: 0.071
Optimizer: RMSProp
Training Accuracy: 0.9934
Validation Accuracy: 0.9649
Training Loss: 0.0201
Validation Loss: 0.0804
Optimizer: Adam
Training Accuracy: 0.989
Validation Accuracy: 0.9912
Training Loss: 0.0344
Validation Loss: 0.0614
0 files and 16 cells to analyze
Spaces: 4
Cell 26 of 27

POST-LAB QUESTIONS (PROVIDE BRIEF ANSWERS TO THE FOLLOWING QUESTIONS)

1. **Which activation function achieved the best performance in your experiment? Justify your answer using the obtained results.**
   The ReLU activation function achieved the best performance in the experiment. It converged faster, reduced the vanishing gradient problem, and produced higher training and validation accuracy compared to Sigmoid and Tanh. The training graphs also showed quicker loss reduction and better overall model performance.

2. **Compare the convergence behavior of SGD, Momentum, RMSProp, and Adam based on the training graphs.**
   SGD showed the slowest convergence with noticeable fluctuations during training. Momentum improved convergence by reducing oscillations and accelerating learning. RMSProp adapted the learning rate for each parameter, resulting in faster convergence. Adam combined the advantages of Momentum and RMSProp, achieving the fastest convergence with the most stable and accurate training performance.

3. **Why does the ReLU activation function generally train faster than the Sigmoid activation function?**
   ReLU trains faster because it has a simple mathematical operation and avoids the vanishing gradient problem for positive inputs. This allows gradients to flow efficiently during backpropagation, leading to quicker learning. In contrast, the Sigmoid function saturates at extreme values, causing very small gradients and slower convergence.

4. **How does version control using GitHub improve the reproducibility and collaboration of deep learning projects?**
   GitHub improves reproducibility by maintaining a complete history of code, datasets, and model changes. It allows developers to track modifications, revert to previous versions, and share projects easily. Collaboration becomes more effective through branching, pull requests, issue tracking, and code reviews, enabling multiple contributors to work efficiently.

5. **If you were asked to train a deep neural network for an image classification task, which activation function and optimizer would you choose? Justify your selection based on the experimental observations.**
   I would choose the ReLU activation function with the Adam optimizer. ReLU enables faster learning by preventing the vanishing gradient problem, while Adam provides adaptive learning rates and stable convergence. Based on the experimental observations, this combination achieved the highest accuracy, fastest convergence, and best overall performance for deep learning tasks.

GitHub:https://github.com/Pradeep-Manikandan-1-D/24BAD088_DEEP_LEARNING_LAB

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
	Faculty Signature	